Lógica de Programação | Aula 13

Dicionários e Imports em Python

João Quentino

18 de Março de 2026

Jovem Tech | Turma 04A - Manhã

Recapitulação

O que já aprendemos:

Estruturas de Dados:

- Listas (ordenadas, indexadas)
- Strings (imutáveis)
- Tipos básicos

Funções:

- def, parâmetros, return
- Escopo de variáveis
- Modularização

Hoje:

- Dicionários: estrutura chave-valor
- Imports: reutilizar código
- Organizar projetos

Agenda de Hoje

Introdução aos Dicionários

Criando e Acessando Dicionários

Métodos de Dicionário

Dicionários Aninhados

Introdução aos Imports

Criando Seus Próprios Módulos

Exercícios Práticos

Projeto Integrador

Boas Práticas

Recapitulação

Introdução aos Dicionários

Por Que Precisamos de Dicionários?

Problema com listas:

Armazenar dados de uma pessoa usando listas:

- `pessoa = ["Ana", 25, "Engenheira"]`
- Qual é o nome? `pessoa[0]` ← Precisa memorizar índices!
- Qual é a idade? `pessoa[1]`
- Qual é a profissão? `pessoa[2]`

Problemas:

- Índices numéricos não são descritivos
- Difícil de lembrar qual índice é qual
- Se mudar a ordem, quebra tudo
- Código confuso

Solução: Dicionários!

Com dicionários, usamos nomes descritivos:

```
pessoa = {
    "nome": "Ana",
    "idade": 25,
    "profissao": "Engenheira"
}
# Acesso descritivo e claro!
print(pessoa["nome"])          # Ana
print(pessoa["idade"])         # 25
print(pessoa["profissao"])     # Engenheira
```

Vantagens:

- Acesso por chave (nome descritivo)
- Código legível e autoexplicativo
- Flexível e poderoso

O que são Dicionários?

Definição:

Dicionário é uma coleção não-ordenada de pares chave:valor.

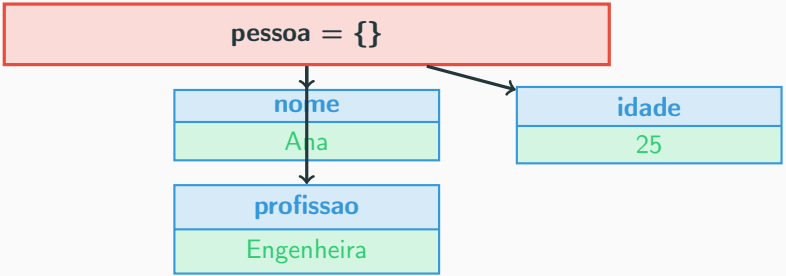
Analogia: Dicionário de Verdade

- Palavra (chave) → Definição (valor)
- "Python"→ "Linguagem de programação"
- "Função"→ "Bloco de código reutilizável"

Características:

- **Chaves únicas**: não pode repetir chave
- **Mutável**: pode modificar após criar
- **Não-ordenado**: não tem índices numéricos (Python 3.7+ mantém ordem de inserção)
- **Chaves imutáveis**: strings, números, tuplas
- **Valores**: qualquer tipo

Visualizando Dicionários



Cada **chave** mapeia para um **valor**

Criando e Acessando Dicionários

Criando Dicionários

Sintaxe: usar chaves { }

```python
# Dicionario vazio
dicionario_vazio = {}

# Dicionario com dados
aluno = {
    "nome": "Carlos",
    "nota": 8.5,
    "aprovado": True
}

# Diferentes tipos de chaves e valores
dados = {
    "texto": "Python",
    1: "um",
    (1, 2): "tupla",
    "lista": [1, 2, 3]
}
```

Acessando Valores

Use colchetes com a chave:

```python
aluno = {
    "nome": "Carlos",
    "nota": 8.5,
    "curso": "Python"
}

# Acesso direto
print(aluno["nome"])    # Carlos
print(aluno["nota"])    # 8.5

# Erro se chave nao existe!
print(aluno["idade"])   # KeyError!

# Metodo get() - mais seguro
print(aluno.get("idade"))            # None
print(aluno.get("idade", 0))         # 0 (valor padrao)
print(aluno.get("nome", "Sem nome"))  # Carlos
```

Modificando Dicionários

Adicionar, modificar e remover:

```python
aluno = {"nome": "Carlos", "nota": 8.5}
# Adicionar nova chave
aluno["idade"] = 20
print(aluno)  # {"nome": "Carlos", "nota": 8.5, "idade": 20}

# Modificar valor existente
aluno["nota"] = 9.0
print(aluno["nota"])  # 9.0

# Remover chave
del aluno["idade"]
print(aluno)  # {"nome": "Carlos", "nota": 9.0}

# pop() - remove e retorna valor
nota = aluno.pop("nota")
print(nota)    # 9.0
print(aluno)   # {"nome": "Carlos"}
```

Métodos de Dicionário

keys(), values() e items()

Acessar chaves, valores ou ambos:

```python
aluno = {"nome": "Ana", "idade": 22, "curso": "Python"}

# keys() - todas as chaves
chaves = aluno.keys()
print(chaves)  # dict_keys(["nome", "idade", "curso"])

# values() - todos os valores
valores = aluno.values()
print(valores)  # dict_values(["Ana", 22, "Python"])

# items() - pares (chave, valor)
itens = aluno.items()
print(itens)
# dict_items([("nome", "Ana"), ("idade", 22), ("curso", "Python")])
```

Iterando sobre Dicionários

Método 1: Iterar sobre chaves (padrão)

```python
aluno = {"nome": "Ana", "idade": 22, "curso": "Python"}

for chave in aluno:
    print(f"{chave}: {aluno[chave]}")
```

Método 2: Iterar sobre valores

```python
for valor in aluno.values():
    print(valor)
```

Método 3: Iterar sobre pares (melhor!)

```python
for chave, valor in aluno.items():
    print(f"{chave}: {valor}")
```

Outros Métodos Úteis

```python
aluno = {"nome": "Carlos", "nota": 8.5}

# update() - mesclar dicionarios
novos_dados = {"idade": 20, "curso": "Python"}
aluno.update(novos_dados)
print(aluno)
# {"nome": "Carlos", "nota": 8.5, "idade": 20, "curso": "Python"}

# clear() - limpar todos os itens
aluno.clear()
print(aluno)  # {}

# copy() - criar copia
original = {"a": 1, "b": 2}
copia = original.copy()
copia["c"] = 3
print(original)  # {"a": 1, "b": 2}
print(copia)     # {"a": 1, "b": 2, "c": 3}
```

Verificando Chaves

Operador in para verificar existência:

```python
aluno = {"nome": "Ana", "idade": 22}

# Verificar se chave existe
if "nome" in aluno:
    print("Nome cadastrado!")

if "email" not in aluno:
    print("Email nao cadastrado")
    aluno["email"] = input("Digite o email: ")

# Forma segura de acessar
chave = "nota"
if chave in aluno:
    print(f"{chave}: {aluno[chave]}")
else:
    print(f"{chave} nao encontrado")
```

Dicionários Aninhados

Dicionários dentro de Dicionários

Estruturas complexas:

```python
turma = {
    "aluno1": {
        "nome": "Ana",
        "notas": [8.5, 9.0, 7.5],
        "aprovado": True
    },
    "aluno2": {
        "nome": "Bruno",
        "notas": [6.0, 7.0, 6.5],
        "aprovado": True
    }
}

# Acessar valores aninhados
print(turma["aluno1"]["nome"])          # Ana
print(turma["aluno2"]["notas"][0])      # 6.0
print(turma["aluno1"]["aprovado"])      # True
```

Lista de Dicionários

Padrão muito comum:

```python
alunos = [
    {"nome": "Ana", "nota": 8.5},
    {"nome": "Bruno", "nota": 7.0},
    {"nome": "Carlos", "nota": 9.0}
]

# Iterar sobre lista de dicionarios
for aluno in alunos:
    print(f"{aluno['nome']}: {aluno['nota']}")

# Calcular media
total = sum(aluno["nota"] for aluno in alunos)
media = total / len(alunos)
print(f"Media da turma: {media:.2f}")
```

Introdução aos Imports

Por Que Importar Código?

Organização e Reutilização

Problemas sem imports:

- Todo código em um arquivo gigante
- Difícil de encontrar funções
- Impossível reutilizar em outros projetos
- Conflitos de nomes

Solução: Módulos

- Dividir código em arquivos separados
- Importar apenas o que precisa
- Reutilizar em vários projetos
- Usar bibliotecas de terceiros

O que são Módulos?

Módulo = arquivo Python (.py)

Tipos de módulos:

1. **Built-in**: vêm com Python
   - `math`, `random`, `datetime`
   - Sempre disponíveis
2. **Terceiros**: instalados separadamente
   - `requests`, `numpy`, `pandas`
   - Instalar com `pip install`
3. **Seus próprios**: que você cria
   - Seus arquivos .py
   - Organizados em pastas

import Básico

Importar módulo completo:

```python
# Importar modulo math
import math

# Usar funcoes do modulo
raiz = math.sqrt(16)
print(raiz)  # 4.0

pi = math.pi
print(pi)    # 3.141592653589793

seno = math.sin(math.pi / 2)
print(seno)  # 1.0
```

Sintaxe: `modulo.funcao()`

from ... import ...

Importar funções específicas:

```
# Importar apenas sqrt e pi
from math import sqrt, pi

# Usar diretamente (sem math.)
raiz = sqrt(16)
print(raiz)  # 4.0

print(pi)    # 3.141592653589793

# Mas outras funcoes nao estao disponiveis
# print(sin(0))  # ERRO! sin nao foi importado
```

Importar tudo (não recomendado):

```
from math import *  # Importa TODAS as funcoes
# Pode causar conflitos de nomes!
```

import ... as ...

Dar apelidos para módulos:

```python
# Apelido comum: pd
import pandas as pd
# Usar com apelido
df = pd.DataFrame()

# Apelido para funcoes longas
from math import factorial as fat

resultado = fat(5)
print(resultado)  # 120
```

Quando usar:

- Módulos com nomes longos
- Convenções da comunidade (numpy → np, pandas → pd)
- Evitar conflitos de nomes

Módulo random

Geração de números aleatórios:

```python
import random

# Numero inteiro aleatorio
dado = random.randint(1, 6)
print(f"Dado: {dado}")

# Numero float aleatorio [0.0, 1.0)
aleatorio = random.random()
print(aleatorio)

# Escolher item aleatorio de lista
frutas = ["maca", "banana", "laranja"]
sorteada = random.choice(frutas)
print(sorteada)
# Embaralhar lista
random.shuffle(frutas)
print(frutas)
```

Módulo datetime

Trabalhar com datas e horas:

```python
from datetime import datetime, date

# Data/hora atual
agora = datetime.now()
print(agora)  # 2026-03-18 14:30:00.123456

# Apenas data
hoje = date.today()
print(hoje)   # 2026-03-18

# Formatar data
print(agora.strftime("%d/%m/%Y"))  # 18/03/2026
print(agora.strftime("%H:%M"))     # 14:30
# Calcular idade
nascimento = date(2000, 5, 15)
idade = (hoje - nascimento).days // 365
print(f"Idade: {idade} anos")
```

Criando Seus Próprios Módulos

Estrutura de Projeto

Organizar código em arquivos:

```
meu_projeto/
    main.py            # Arquivo principal
    calculos.py        # Funcoes matematicas
    validacoes.py      # Funcoes de validacao
```

calculos.py:

```python
def somar(a, b):
    return a + b

def calcular_media(numeros):
    return sum(numeros) / len(numeros)

PI = 3.14159
```

Importando Seus Módulos

main.py:

```python
# Importar modulo proprio
import calculos
# Usar funcoes
resultado = calculos.somar(5, 3)
print(resultado)  # 8

notas = [8.5, 7.0, 9.0]
media = calculos.calcular_media(notas)
print(media)  # 8.166666...

print(calculos.PI)  # 3.14159

# Ou importar funcoes especificas
from calculos import somar, PI

print(somar(10, 20))  # 30
print(PI)             # 3.14159
```

Exemplo Completo

validacoes.py:

```python
def validar_cpf(cpf):
    """Valida CPF simplificado."""
    return len(cpf) == 11 and cpf.isdigit()

def validar_email(email):
    """Valida email simplificado."""
    return "@" in email and "." in email
```

main.py:

```python
from validacoes import validar_cpf, validar_email

cpf = input("CPF: ")
if validar_cpf(cpf):
    print("CPF valido!")
else:
    print("CPF invalido!")
```

Exercícios Práticos

Exercício 1: Agenda de Contatos

Desafio (15 min):

Crie uma agenda usando dicionários:

```
agenda = {}

# Funcionalidades:
# 1. Adicionar contato (nome, telefone, email)
# 2. Buscar contato por nome
# 3. Listar todos os contatos
# 4. Remover contato
# 5. Sair

# Estrutura de cada contato:
# {
#     "nome": {"telefone": "123456", "email": "email@example.com"}
# }
```

Exercício 2: Contador de Palavras

Desafio (15 min):

Conte quantas vezes cada palavra aparece em um texto:

```python
def contar_palavras(texto):
    """
    Retorna dicionario com contagem de palavras.

    Exemplo:
    texto = "python e legal python e poderoso"
    resultado = {"python": 2, "e": 2, "legal": 1, "poderoso": 1}
    """
    # Complete...

# Teste
texto = "o rato roeu a roupa do rei de roma"
contagem = contar_palavras(texto)
print(contagem)
```

Exercício 3: Sistema de Notas com Dicionário

Desafio (20 min):

Refatore sistema de notas usando dicionários:

```
alunos = {}

# Cada aluno:
# {
#     "nome": {
#         "notas": [8.5, 7.0, 9.0],
#         "faltas": 2
#     }
# }

# Funcoes:
# - adicionar_aluno(nome)
# - adicionar_nota(nome, nota)
# - calcular_media(nome)
# - verificar_aprovacao(nome)  # Media >= 7 e faltas < 5
```

Exercício 4: Tradutor Simples

Desafio (10 min):

Crie tradutor português-inglês:

```python
traducoes = {
    "ola": "hello",
    "mundo": "world",
    "python": "python",
    "programacao": "programming",
    "computador": "computer"
}

def traduzir(palavra):
    """Traduz palavra ou retorna 'Traducao nao encontrada'."""
    # Complete...

# Teste
print(traduzir("ola"))          # hello
print(traduzir("python"))       # python
```

Exercício 5: Módulo de Utilidades

Desafio (20 min):

Crie arquivo `utilidades.py` com funções:

```python
# utilidades.py
def eh_par(numero):
    """Verifica se numero e par."""
    # Complete...

def eh_primo(numero):
    """Verifica se numero e primo."""
    # Complete...

def fatorial(n):
    """Calcula fatorial de n."""
    # Complete...
```

Depois importe e use em outro arquivo!

Projeto Integrador

Projeto: Sistema de Biblioteca

Crie sistema completo de gerenciamento de biblioteca.

Estrutura de arquivos:

```
biblioteca/
    main.py
    livros.py       # Funcoes de gerenciamento
    relatorios.py   # Funcoes de relatorio
```

Estrutura de dados:

```python
biblioteca = {
    "livro1": {
        "titulo": "Python Basico",
        "autor": "Autor A",
        "ano": 2020,
        "disponivel": True
    }
```

Projeto: Funcionalidades

livros.py:

```python
def adicionar_livro(biblioteca, id, titulo, autor, ano):
    """Adiciona livro a biblioteca."""
    pass

def emprestar_livro(biblioteca, id):
    """Marca livro como indisponivel."""
    pass

def devolver_livro(biblioteca, id):
    """Marca livro como disponivel."""
    pass

def buscar_por_autor(biblioteca, autor):
    """Retorna lista de livros do autor."""
    pass
```

Projeto: Relatórios

relatorios.py:

```python
def listar_disponiveis(biblioteca):
    """Lista todos os livros disponiveis."""
    for id, dados in biblioteca.items():
        if dados["disponivel"]:
            print(f"{dados['titulo']} - {dados['autor']}")

def estatisticas(biblioteca):
    """Exibe estatisticas da biblioteca."""
    total = len(biblioteca)
    disponiveis = sum(1 for l in biblioteca.values()
                      if l["disponivel"])
    print(f"Total de livros: {total}")
    print(f"Disponiveis: {disponiveis}")
    print(f"Emprestados: {total - disponiveis}")
```

Projeto: Main

main.py:

```python
import livros
import relatorios
biblioteca = {}

while True:
    print("\n=== BIBLIOTECA ===")
    print("1 - Adicionar livro")
    print("2 - Emprestar livro")
    print("3 - Devolver livro")
    print("4 - Listar disponiveis")
    print("5 - Estatisticas")
    print("0 - Sair")
    opcao = input("Escolha: ")

    if opcao == "1":
        # Implementar adicionar...
        pass
    # Continue...
```

Boas Práticas

Boas Práticas com Dicionários

Diretrizes essenciais:

1. **Use get() para acesso seguro**
   - Evita KeyError
   - Permite valor padrão
2. **Chaves descritivas**
   - `"nome_completo"` melhor que `"nc"`
   - Use strings como chaves
3. **Verifique existência com in**
   - Antes de acessar ou modificar
4. **Use items() para iterar**
   - Mais eficiente que acessar separadamente

Boas Práticas com Imports

Diretrizes essenciais:

1. **Imports no topo do arquivo**
   - Facilita identificar dependências
2. **Ordem dos imports**
   - 1. Biblioteca padrão (math, random)
   - 2. Terceiros (requests, pandas)
   - 3. Seus módulos (calculos, validacoes)
3. **Evite import ***
   - Causa conflitos de nomes
   - Dificulta saber origem das funções
4. **Use nomes descritivos para arquivos**
   - `calculos.py`, não `utils.py`

Recapitulação

O que Aprendemos Hoje

Dicionários e Imports:

Dicionários:

1. Pares chave-valor
2. Criar com { }
3. Acessar com [ ]
4. get() seguro
5. keys(), values(), items()
6. Iterar com for
7. Aninhamento

Imports:

1. import modulo
2. from ... import ...
3. import ... as ...
4. Módulos built-in
5. random, datetime
6. Criar próprios módulos
7. Organizar projetos

Dicionários vs Listas

Característica	Lista	Dicionário
Acesso	Por índice (0, 1, 2...)	Por chave (nome, idade...)
Ordenado	Sim	Sim* (Python 3.7+)
Mutável	Sim	Sim
Duplicatas	Permite	Chaves únicas
Uso	Sequências	Mapeamentos
Exemplo	`[1, 2, 3]`	`{"a": 1, "b": 2}`

Quando usar cada um:

- **Lista**: sequência de itens similares
- **Dicionário**: dados estruturados com rótulos

Próximos Passos

O que vem pela frente:

- **Tuplas e Sets**: Outras estruturas de dados
- **Tratamento de Exceções**: try/except
- **Arquivos**: Ler e escrever dados
- **Orientação a Objetos**: Classes e objetos
- **Projetos Finais**: Integrar tudo!

Continue praticando! Dicionários são extremamente úteis.

Desafio para Casa

Projetos para praticar:

1. **Sistema de Estoque:**
   - Dicionário de produtos
   - Adicionar, remover, atualizar
   - Relatórios de estoque baixo
2. **Jogo de RPG:**
   - Personagem em dicionário (HP, ataque, defesa)
   - Inimigos em dicionário
   - Sistema de combate
3. **Conversor de Moedas:**
   - Dicionário de taxas de câmbio
   - Módulo separado para conversões
   - Menu interativo

Dúvidas e Encerramento

Momento para perguntas!

Contato:

joao.quentino@dcomp.ufs.br

Obrigado!